AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Amrita School of AI
COIMBATORE CAMPUS

HIGH-PERFORMANCE AND CLOUD COMPUTING

Unit 4: Cloud Computing and the HPC-AI Convergence

What the cloud actually is, when it beats owning a cluster, and why training a large model is an HPC problem wearing different clothes.

Prepared by **Dr. Abhijith Anandakrishnan**
Assistant Professor, Amrita School of AI

COURSE
23AID304 High-Performance and Cloud Computing

UNIT
4 of 4

SYLLABUS
Cloud computing and its importance, benefits and challenges, IaaS PaaS SaaS, cloud architecture, storage, networking, security, applications, HPC and AI synergy, training and inference using HPC

COMPANION COURSE
24AIM332 Introduction to Cloud Computing

OFFERING
Odd Semester 2026-27

Part I: Cloud computing

What the cloud actually is

Strip away the marketing and cloud computing is one idea: **someone else owns the hardware, and you rent it by the minute through an API.**

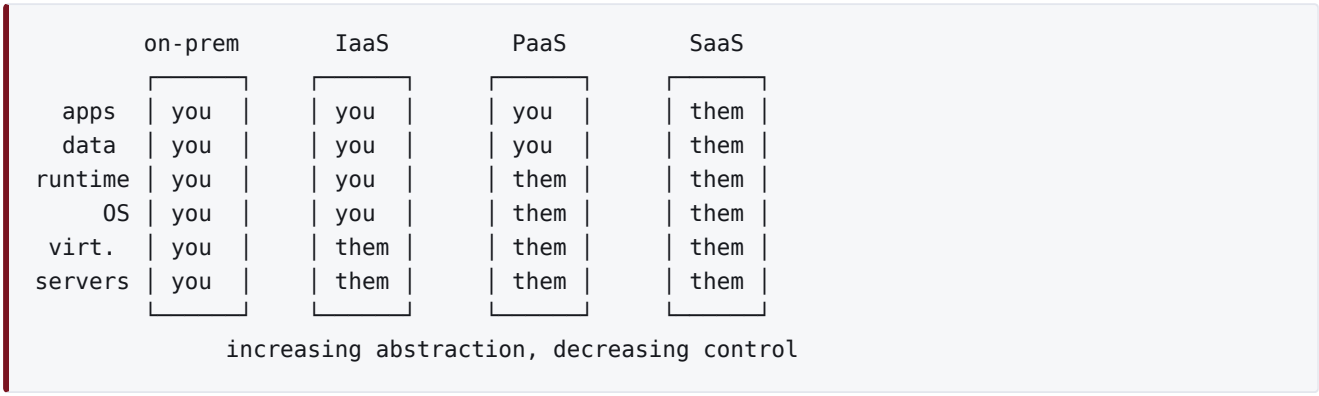
Everything else follows. You do not buy a server, wait six weeks for delivery, rack it, cool it, and depreciate it over five years. You issue an API call and a machine exists thirty seconds later. You issue another and it is gone, and you stop paying.

> **DEFINITION · THE FIVE ESSENTIAL CHARACTERISTICS (NIST)**
>
> **On-demand self-service**: provision without human interaction. **Broad network access**: reachable over standard protocols. **Resource pooling**: physical resources shared between tenants. **Rapid elasticity**: scale out and back quickly, apparently without limit. **Measured service**: metered usage, pay for what you consume.
>
> If something lacks these, it is hosting, not cloud. The distinction matters: a rented dedicated server you keep for a year is neither elastic nor measured.

Service models

The standard three, distinguished by **where the line of responsibility falls**.

	YOU MANAGE	PROVIDER MANAGES	EXAMPLE
On-premises	Everything	Nothing	`asaicompute`
IaaS	OS, runtime, application, data	Virtualisation, servers, storage, network	EC2, Azure VM, GCE
PaaS	Application, data	Everything below	App Service, App Engine, Heroku
SaaS	Configuration only	Everything	Gmail, Salesforce, Office 365



Two more worth knowing, both now more common than classical PaaS:

- **CaaS**, containers as a service: you supply a container image, the provider runs and scales it. Kubernetes services, ECS, Cloud Run.
- **FaaS**, functions as a service, or *serverless*: you supply a function, the provider runs it per request and bills per invocation. Lambda, Azure Functions. “Serverless” does not mean no servers; it means no servers *you* think about.

Deployment models

MODEL	WHAT IT IS	WHEN IT FITS
Public	Shared infrastructure, multi-tenant	Variable demand, no data-residency constraint
Private	Dedicated to one organisation	Regulated data, steady predictable load
Hybrid	Both, connected	Steady base on-premises, bursts to public
Community	Shared by organisations with common concerns	Research consortia, government sectors

The hybrid pattern is exactly how a university uses cloud sensibly: routine teaching and research runs on `asaicompute`, and a deadline-driven burst that needs sixty GPUs for a week is rented.

Benefits, honestly stated

- **No capital expenditure.** Rent instead of buy. For a startup or a short project this is decisive.
- **Elasticity.** Match capacity to demand rather than to peak forecast.
- **Speed.** Minutes to provision instead of months to procure.
- **Managed services.** Someone else patches the database and handles its backups.
- **Geographic reach.** Deploy near your users without owning a building there.

- **Access to hardware you could not justify buying.** Try eight H100s for four hours for the price of a meal.

Challenges, equally honestly

PITFALL · CLOUD IS NOT AUTOMATICALLY CHEAPER

For a **steady, predictable, high-utilisation** workload, owning is usually cheaper over three years. Cloud wins on *variable* demand, where you would otherwise buy for the peak and idle most of the time.

A concrete comparison. A GPU node with 4 A100s costs roughly USD 100 000 and lasts four years, so about USD 2.90 per hour if it runs continuously. The equivalent cloud instance is roughly USD 12 to 15 per hour on demand. If your utilisation is above roughly 25 percent, owning wins on hardware cost alone; below that, renting does. Then add power, cooling, space, and the salary of whoever administers it, which is the part institutional comparisons usually forget.

This is exactly the calculation behind `asaicompute` existing at all.

Other real challenges:

- **Egress charges.** Moving data *in* is free; moving it *out* is billed, often at USD 0.08 to 0.12 per GB. Ten terabytes of results costs about USD 1 000 to retrieve. This asymmetry is deliberate and it is the main mechanism of lock-in.
- **Vendor lock-in.** Proprietary managed services are convenient and hard to leave.
- **Latency and bandwidth.** Physics does not care about your provider. Mumbai to Virginia is about 180 ms round trip, and no amount of money changes it.
- **Compliance and data residency.** Where the data physically sits has legal consequences.
- **Shared responsibility.** The provider secures the cloud; you secure what you put in it. Most publicised “cloud breaches” are misconfigured customer resources, not provider failures.
- **Noisy neighbours and variable performance.** Shared hardware means your benchmark today may not reproduce tomorrow, which matters for HPC.

Cloud architecture components

COMPONENT	WHAT IT IS	TYPICAL SERVICES
Compute	VMs, containers, functions	EC2, AKS, Lambda
Storage	Object, block, file	S3, EBS, EFS
Network	Virtual networks, load balancers, DNS	VPC, ALB, Route 53
Identity	Who may do what	IAM, Entra ID
Database	Managed relational, NoSQL, warehouse	RDS, DynamoDB, BigQuery
Observability	Metrics, logs, traces	CloudWatch, Azure Monitor

Three concepts underpin all of it:

DEFINITION · REGION, AVAILABILITY ZONE, EDGE

A **region** is a geographic area, for example `ap-south-1` in Mumbai. An **availability zone** is one or more datacentres within a region, isolated for power and cooling but connected by low-latency links. An **edge location** is a small point of presence used for content delivery.

Design rule: spread across **zones** for availability, across **regions** for disaster recovery and data residency. Zones are cheap to span, regions are not.

Storage types

TYPE	LOOKS LIKE	LATENCY	GOOD FOR	NOT FOR
Object	A key-value store of blobs, HTTP API	~100 ms	Datasets, checkpoints, archives	Anything needing a filesystem
Block	A raw disk attached to one VM	<1 ms	OS disks, databases	Sharing between machines
File	NFS or SMB share	~1 ms	Shared home directories, legacy apps	Extreme throughput

Object storage is the one worth understanding properly, because it behaves differently from a filesystem:

- Flat namespace. “Directories” are a prefix convention, not real.
- Objects are immutable: you replace, you do not edit in place.
- Eventually consistent historically, strongly consistent on major providers now.
- Storage classes trade retrieval time and cost: hot, cool, archive. Archive is very cheap to store and slow and expensive to read.

PITFALL · OBJECT STORAGE IS NOT A PARALLEL FILESYSTEM

Pointing an HPC job at object storage as if it were a POSIX filesystem gives terrible performance: every small read is an HTTP request with ~100 ms latency. The correct pattern is to stage data down to fast local or parallel storage once, compute, and write results back. This is why `asaicompute` has GlusterFS for working data.

Cloud networking

A **virtual private cloud (VPC)** is your own isolated network: address ranges, subnets, routing tables, gateways. Public subnets reach the internet; private subnets do not, and reach out only through a NAT gateway.

Traffic control has two layers:

- **Security groups**: stateful, attached to instances, allow rules only. Return traffic is automatically permitted.
- **Network ACLs**: stateless, attached to subnets, allow and deny rules. Both directions must be stated explicitly.

The hard parts of cloud networking are the ones physics imposes: latency between regions, bandwidth costs, and the fact that a virtualised network adds overhead that matters enormously to MPI and not at all to a web application.

Cloud security

DEFINITION · THE SHARED RESPONSIBILITY MODEL

The provider is responsible for security **of** the cloud: physical datacentres, hypervisor, network fabric, managed service internals. You are responsible for security **in** the cloud: your data, access control, configuration, patching your VMs, encryption choices.

Nearly every large "cloud breach" reported in the press is a customer-side misconfiguration, most often an object storage bucket made public.

The practices that actually matter:

- **Least privilege.** Grant the minimum permission needed. No shared root accounts.
- **Multi-factor authentication** everywhere, without exception on admin accounts.
- **Encrypt at rest and in transit.** Both are close to free now; there is no argument for skipping them.
- **Never commit secrets.** Use a secrets manager. A key in a public repository is scanned and abused within minutes.
- **Log and audit.** You cannot investigate what you did not record.
- **Private by default.** Public network exposure should be a deliberate decision.

ON ASAICOMPUTE · THIS APPLIES TO ASAICOMPUTE TOO

The same principles govern the cluster: individual accounts rather than shared logins, SSH keys rather than passwords where possible, no credentials in submitted scripts, and job output written to your own directory. A cluster is a private cloud with a scheduler in front of it.

HPC in the cloud

Can you run this course's workloads on cloud? Yes, with caveats that are themselves instructive.

ASPECT	ON-PREMISES CLUSTER	CLOUD
Interconnect	InfiniBand, ~1 µs latency	Varies; special HPC instance types needed for anything comparable
Cost model	Capital, then near-zero marginal	Operational, per hour
Scaling	Fixed size	Effectively unlimited
Queue wait	Can be hours	Minutes, if capacity exists
Performance consistency	High	Variable, unless using bare-metal instances
Best for	Steady load, tightly coupled MPI	Bursts, embarrassingly parallel work, experimentation

The decisive technical point is the **interconnect**. Tightly coupled MPI work is latency-sensitive, and ordinary cloud virtual networking has both higher and more variable latency than InfiniBand. Providers sell specialised HPC instances with 100+ Gb/s low-latency fabrics (AWS EFA, Azure InfiniBand) to address this, at a price. Loosely coupled work, parameter sweeps, Monte Carlo, hyperparameter search, runs beautifully on ordinary instances because it barely communicates.

DEFINITION · SPOT AND PREEMPTIBLE INSTANCES

Unused capacity sold at a 60 to 90 percent discount, reclaimable at a few minutes' notice. Excellent for checkpointed or restartable work, useless for a twelve-hour job that cannot be interrupted. Checkpointing your code turns a large discount into free money, which is the main reason serious cloud HPC codes checkpoint aggressively.

Part II: The HPC and AI convergence

Why these two fields merged

For thirty years HPC meant simulation: weather, fluids, molecules, structures. Machine learning was a separate, smaller world.

They collided because training a large neural network turns out to be *exactly* an HPC problem:

- Dominated by dense linear algebra, which is what HPC has optimised since the 1970s.
- Requires more compute than one machine has, so it must be distributed.
- Bound by memory bandwidth and interconnect, the two classic HPC limits.
- Needs the same tools: MPI-style collectives, job schedulers, parallel filesystems.

The traffic runs both ways. Simulation now uses learned surrogate models to replace expensive solver steps, and AI training uses techniques HPC developed decades earlier.

> **DEFINITION · THE CONVERGENCE, IN ONE LINE**
>
> **Training a large model is a distributed dense linear algebra problem with a collective communication step in the inner loop.** Every performance idea in this course applies to it unchanged.

Where the time actually goes

A training step is:

1. Load a batch of data.
2. **Forward pass**: a sequence of matrix multiplications and elementwise operations.
3. **Backward pass**: roughly twice the cost of the forward pass.
4. **Gradient synchronisation** across devices.
5. Optimiser update.

Steps 2 and 3 are cuBLAS and cuDNN work from Unit 3. Step 4 is the distributed part, and it is where the HPC knowledge earns its keep.

Data parallelism

The default strategy: every GPU holds a **complete copy** of the model and processes a different slice of the batch. After the backward pass, gradients are averaged across all GPUs, so every replica applies the same update.

That averaging is an **all-reduce**. It is Unit 2's `MPI_Allreduce`, over gradient tensors, executed thousands of times per hour.

DEFINITION · RING ALL-REDUCE

Arrange P GPUs in a ring. Split the gradient into P chunks. In $P-1$ steps each GPU sends one chunk to its neighbour and accumulates the one it receives (reduce-scatter); in another $P-1$ steps the completed chunks circulate (allgather).

Each GPU sends $2(P-1)/P \times N$ bytes, which tends to $2N$ and is **independent of *P***. That is why ring all-reduce scales: the per-GPU communication volume does not grow with the number of GPUs, only the number of steps does.

NVIDIA's **NCCL** implements this and several other algorithms, choosing by message size and topology, and using NVLink within a node and InfiniBand between nodes. It is the MPI of deep learning, and the ring topology from Unit 1's interconnect table is why.

WORKED EXAMPLE · IS THE NETWORK GOING TO BE YOUR BOTTLENECK?

A 7-billion-parameter model in FP16 has gradients of 7e9 × 2 bytes = **14 GB**. Ring all-reduce moves about 2 × 14 = 28 GB per GPU per step.

Over NVLink at 600 GB/s: 28 / 600 ≈ **47 ms**. Over 100 Gb/s InfiniBand (12.5 GB/s): 28 / 12.5 ≈ **2.2 s**.

If a training step's computation takes about 200 ms, then within a node communication overlaps nicely and costs little; across nodes on 100 Gb/s it dominates by an order of magnitude and the GPUs sit idle.

This single calculation explains gradient compression, gradient accumulation, overlapping communication with the backward pass, and why large training runs are built on the fastest interconnect money can buy.

When the model does not fit

Data parallelism assumes each GPU can hold the whole model. Beyond roughly 10 billion parameters it cannot, and the model itself must be split.

STRATEGY	WHAT IS SPLIT	COMMUNICATION	COST
Data parallel	The batch	All-reduce of gradients per step	Simple, memory-heavy
Tensor parallel	Individual matrices, across GPUs	All-reduce **inside** each layer	Very chatty; keep within a node
Pipeline parallel	Layers, into stages	Activations between adjacent stages	Introduces idle "bubbles"
Sharded (ZeRO / FSDP)	Optimiser state, gradients, parameters	Gather parameters just before use	Large memory saving, more communication

Real systems combine all of them, which the field calls **3-D parallelism**: tensor parallel within a node over NVLink, pipeline parallel across a small group of nodes, data parallel across the rest.

WORKED EXAMPLE · MEMORY ARITHMETIC FOR A 7B MODEL WITH ADAM

Per parameter, in mixed-precision training:

ITEM	BYTES
FP16 weights	2
FP16 gradients	2
FP32 master weights	4
Adam first moment	4
Adam second moment	4
Total	**16**

7e9 × 16 = **112 GB**, before a single activation is stored. An 80 GB A100 cannot hold it. This is precisely why ZeRO and FSDP exist: they shard the optimiser state, gradients and parameters across the data-parallel group, so each GPU stores only its fraction and gathers the rest momentarily when needed.

Mixed precision

Using FP16 or BF16 instead of FP32 halves memory traffic and lets tensor cores do the arithmetic. Since almost every step of training is bandwidth bound, halving the bytes is close to doubling the speed.

FORMAT	BITS	RANGE	NOTES
FP32	32	wide	The reference
TF32	19 stored as 32	FP32 range, FP16 precision	Automatic on tensor cores
FP16	16	narrow	Needs loss scaling to avoid gradient underflow
BF16	16	FP32 range, less precision	No loss scaling needed; now the default
FP8	8	very narrow	Hopper and later; used for the largest models

PITFALL · REDUCED PRECISION CHANGES YOUR ANSWERS

In Unit 2 we noted floating-point addition is not associative, so a parallel sum differs slightly from a serial one. In mixed precision the effect is far larger, and it interacts with data parallelism: change the number of GPUs and the gradient reduction happens in a different order, so training is not bitwise reproducible. For scientific work where reproducibility matters, this must be a conscious decision, not a surprise.

Inference is a different problem

Training and inference have opposite characteristics, and confusing them leads to bad engineering.

	TRAINING	INFERENCE
Runs	Once, for weeks	Constantly, for years
Metric	Throughput	Latency, and cost per request
Batch	Large	Often one
Precision	FP16/BF16 with FP32 master	INT8, FP8, or lower
Bound by	Compute and interconnect	Memory bandwidth, almost always
Hardware	Biggest GPUs available	Whatever meets latency at least cost

WORKED EXAMPLE · WHY TOKEN GENERATION IS MEMORY BOUND

Generating one token from a 7B model in FP16 requires reading all 14 GB of weights once, and performs about $2 \times 7e9 = 14$ GFLOP of arithmetic.

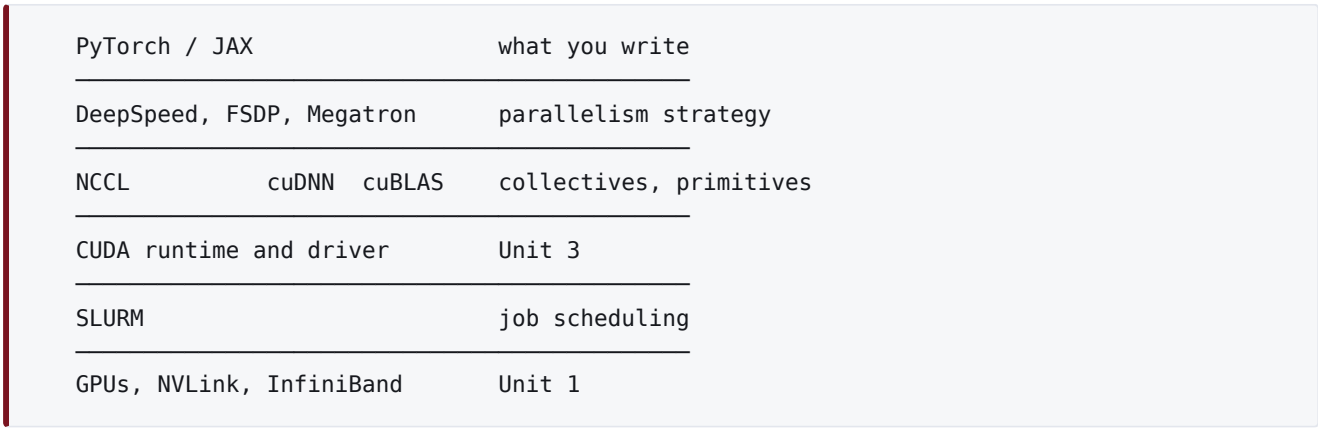
Arithmetic intensity = 14e9 FLOP / 14e9 bytes = **1 FLOP per byte**.

An A100's ridge point is around 100 FLOP/byte. At $I = 1$ we are two orders of magnitude to the left of it: hopelessly memory bound. The GPU's arithmetic units are almost entirely idle, waiting for weights.

Everything in modern inference optimisation follows from this one number: **quantisation** (fewer bytes per weight), **batching** (reuse the weights across many requests, raising I directly), **KV caching** (avoid recomputation), **speculative decoding** (verify several tokens per weight sweep). Each is an attempt to move right on the roofline.

This is Unit 1's model, applied to the most commercially important computation of the decade, and giving the correct answer.

The software stack



Every layer of that stack is something this course has covered. That is the point of putting this unit last.

ON ASAICOMPUTE · A MULTI-GPU TRAINING JOB ON ASAICOMPUTE

```bash
# !/bin/bash
```

SBATCH --job-name=train

SBATCH --nodes=2

SBATCH --ntasks-per-node=2 # one task per GPU

SBATCH --gres=gpu:2

SBATCH --cpus-per-task=24

SBATCH --time=04:00:00

export MASTER_ADDR=$(scontrol show hostnames “$SLURM_JOB_NODELIST” | head -n1) export MASTER_PORT=29500 export NCCL_DEBUG=INFO # prints the topology NCCL chose

srun python train.py --distributed ```

Four A100s across two nodes. `NCCL_DEBUG=INFO` is worth setting the first time: it reports which transport NCCL selected, and discovering that it fell back to TCP sockets instead of using the fast fabric explains a great many disappointing scaling results.

Self-assessment

1. State the five NIST essential characteristics and explain why a rented dedicated server for a year is not cloud. 2. Draw the responsibility boundary for IaaS, PaaS and SaaS. Where does a managed Kubernetes service sit? 3. Give a workload for which owning hardware is cheaper than cloud, and one for which the reverse holds. Justify both. 4. What are egress charges and why do they matter strategically? 5. Distinguish region, availability zone and edge location. What do you spread across each, and why? 6. Compare object, block and file storage. Why is object storage a poor direct backing store for an HPC job? 7. Explain the difference between a security group and a network ACL. 8. State the shared responsibility model. Who is at fault for a public storage bucket leaking data? 9. Why is tightly coupled MPI work harder to run well in the cloud than an embarrassingly parallel parameter sweep? 10. What are spot instances and what property must a job have to use them well? 11. Explain in one sentence why training a large model is an HPC problem. 12. Describe ring all-reduce. Why is per-GPU communication volume independent of the number of GPUs? 13. A 13B-parameter model trains in FP16. Compute the gradient size and the bytes each GPU sends per all-reduce step. 14. Compute the memory needed for a 13B model with Adam in mixed precision. Will it fit on one 80 GB A100? What technique addresses this? 15. Distinguish data, tensor, and pipeline parallelism. Which should stay inside a single node, and why? 16. Why is BF16 usually preferred over FP16 for training despite having less precision? 17. Why is distributed training not bitwise reproducible when the GPU count changes? 18. Compute the arithmetic intensity of single-token generation from a 13B FP16 model. Is it memory or compute bound? 19. Using your answer to 18, explain why batching improves inference throughput so much, in roofline terms. 20. Name each layer of the training software stack and the unit of this course where you met the underlying idea.

---

1. On-demand self-service, broad network access, resource pooling, rapid elasticity, measured service. A dedicated server rented for a year fails rapid elasticity (fixed capacity for the term) and measured service (a flat fee rather than metered consumption), so it is hosting. **2.** IaaS: provider handles virtualisation and below, you handle OS upward. PaaS: provider handles runtime and below, you handle application and data. SaaS: provider handles everything, you configure. Managed Kubernetes sits between IaaS and PaaS: the control plane is managed, but you still manage node images, scaling and workloads. **3.** Owning is cheaper for a steady, high-utilisation workload such as a department's continuous simulation and teaching load, because the hardware is amortised over four years and marginal cost is near zero above roughly 25 percent utilisation. Cloud is cheaper for a burst, such as needing sixty GPUs for one week before a conference deadline, where buying for the peak would leave the hardware idle for eleven months. **4.** Charges for data leaving the provider's network, typically USD 0.08 to 0.12 per GB, while ingress is free. Strategically they create lock-in: accumulating tens of terabytes makes migrating to another provider expensive in itself, independently of any technical difficulty. **5.** A region is a geographic area; an availability zone is an isolated datacentre grouping within a region; an edge location is a small point of presence for content delivery. Spread across zones for availability, because they fail independently but are cheap to span; spread across regions for disaster recovery and data residency, accepting higher latency and transfer cost. **6.** Object storage is a flat key-value store of immutable blobs accessed over HTTP with ~100 ms latency. Block storage is a raw disk attached to one machine with sub-millisecond latency. File storage is a shared POSIX-like filesystem. Object storage is a poor direct backing store because each small read is an HTTP round trip; the correct pattern is to stage the dataset onto fast local or parallel storage once and compute from there. **7.** A security group is stateful, attaches to instances, and holds allow rules only, with return traffic automatically permitted. A

network ACL is stateless, attaches to subnets, supports both allow and deny rules, and requires both directions to be stated explicitly. **8.** The provider secures the cloud (physical infrastructure, hypervisor, managed service internals); the customer secures what they put in it (data, access control, configuration). A public bucket is a customer configuration error, so the customer is at fault. **9.** Tightly coupled MPI is sensitive to interconnect latency and its variability, and ordinary cloud virtual networking has both higher and less predictable latency than InfiniBand; specialised HPC instance types are needed and cost more. An embarrassingly parallel sweep barely communicates, so network quality is almost irrelevant and ordinary instances work well. **10.** Spare capacity sold at a 60 to 90 percent discount but reclaimable at a few minutes' notice. The job must be checkpointed or otherwise restartable, so that reclamation costs only the work since the last checkpoint. **11.** Because it is a distributed dense linear algebra computation, larger than one machine, limited by memory bandwidth and interconnect, with a collective reduction in its inner loop. **12.** *P* GPUs form a ring; the gradient is split into *P* chunks; in *P*−1 reduce-scatter steps each GPU forwards and accumulates one chunk, then in *P*−1 allgather steps the finished chunks circulate. Each GPU sends 2(*P*−1)/*P* × *N* bytes, which approaches 2*N* and does not grow with *P*; only the number of steps grows. **13.** Gradients: 13e9 × 2 bytes = **26 GB**. Each GPU sends about 2 × 26 = **52 GB** per step. **14.** 13e9 × 16 bytes = **208 GB**, excluding activations. It does not fit on an 80 GB A100. ZeRO or FSDP addresses this by sharding optimiser state, gradients and parameters across the data-parallel group, so each GPU stores only its share and gathers parameters transiently when needed. **15.** Data parallelism splits the batch, with one all-reduce per step. Tensor parallelism splits individual weight matrices, requiring an all-reduce inside every layer. Pipeline parallelism splits layers into stages, passing activations between adjacent stages and introducing idle bubbles. Tensor parallelism should stay within a node, because its communication is frequent and latency-sensitive and needs NVLink-class bandwidth. **16.** BF16 keeps FP32's exponent range while giving up mantissa bits, so gradients do not underflow and no loss scaling is needed. FP16 has a narrow range and requires careful loss scaling to train stably, which is an extra source of failure. **17.** Floating-point addition is not associative, and the gradient all-reduce combines partial sums in an order determined by the number of GPUs and the chosen collective algorithm. Changing the GPU count changes the summation order and so changes the low-order bits, which then diverge over many steps. **18.** Weights read: 13e9 × 2 = 26 GB. Arithmetic: about 2 × 13e9 = 26 GFLOP. *I* = 26e9 / 26e9 = **1 FLOP per byte**, against a ridge point near 100, so it is severely **memory bound**. **19.** Batching reuses the same weight sweep for many sequences, so the bytes read stay at 26 GB while the arithmetic multiplies by the batch size. *I* rises roughly in proportion to batch size, moving the kernel to the right along the sloped roof towards the compute-bound region, which is where the hardware's arithmetic capacity finally gets used. **20.** PyTorch/JAX, the application layer. DeepSpeed/FSDP/Megatron, parallelism strategy, which is Unit 1's decomposition. NCCL, collective communication, which is Unit 2's MPI collectives. cuBLAS and cuDNN, dense linear algebra, from Unit 3. CUDA runtime and driver, Unit 3. SLURM, job scheduling, Units 0 and 2's cluster work. GPUs, NVLink and InfiniBand, Unit 1's architecture and interconnects.

Closing

Four units, one argument.

Unit 0 said the machine stopped getting faster, so you must use more of it at once. Unit 1 gave you the means to say how much faster it could possibly go. Unit 2 delivered that on cores and nodes, Unit 3 on accelerators, and Unit 4 showed that the most prominent computational problem of the moment, training and serving large models, is the same problem again: dense linear algebra, limited by data movement, distributed across a machine too big to program by accident.

If you remember one thing, make it the roofline question: *what is the ceiling, and how close am I to it?* Almost every performance decision in your career will be a version of that question.